Search and Retrieve Robot

Ben Stone & Brandt Bolduc

WEBER STATE UNIVERSITY
Engineering, Applied Science & Technology

DEPARTMENT OF
ELECTRICAL & COMPUTER
ENGINEERING

December 12, 2025

Contents

1 Introduction **2**
1.1 Project Goal 2

2 Scope **2**

3 Project Status **3**

4 Design Overview **3**
4.1 System Components & Hardware Description 3
4.2 System Architecture & ROS Nodes 4
4.3 Theory of Operation 5
4.3.1 Top Block Diagram 5
4.4 RQT Graph 5

5 Conclusion **6**
5.1 Challenges 6
5.2 Future Improvements 6

Appendices **8**

A Appendix **8**
A.1 Circuit Schematics 8
A.1.1 Arduino 8
A.1.2 USB Hub 9

1 Introduction

ROS2 Jazzy Source GitHub

For ECE6730/5730, the final project was to design, build and demonstrate an autonomous operational robot. Each student or group was to develop their own idea of what they would like their robot to do and propose their idea to the instructor.

The proposed project dictated in this document was to design and build a robot to autonomously map a room, while exploring the unknown area if an object (with an april tag) was discovered it would stop to pick up and retrieve the object back to the original starting position.

To complete this goal, a modular software architecture was implemented using the Robot Operating System (ROS 2 Jazzy). This framework enabled the separation of key functions—sensor processing, navigation logic, and actuator control—into interoperable nodes. A Raspberry Pi 5 serves as the primary ROS master, running the high-level autonomy stack. An Arduino Mega handles low-level hardware interfacing, including motor control via a Sabertooth dual motor driver, and direction and speed reporting via the IMU. The sensor suite consists of a 360-degree RPLidar A2M8, an IMU for orientation data, and two USB cameras: one front-facing for initial object detection and one mounted on the arm. All components are integrated onto a custom two-tread, differentially-steered mobile chassis.

1.1 Project Goal

The primary goal was to develop a fully autonomous robot capable of exploring an unknown environment, identifying a specific target object marked with an AprilTag, retrieving the object with an onboard arm, and returning it to the starting location. The project aimed to integrate mapping, navigation, computer vision, and manipulator control into a single, cohesive system.

2 Scope

This document provides a comprehensive overview of the autonomous retrieval robot project. Its scope is defined as follows:

- Project Goal & Completion: A statement of the initial project objectives and a summary of the robot's final, demonstrated capabilities compared to those goals.
- System Architecture: A high-level description of the software architecture, including a short summary of each major ROS 2 node, its purpose, and its role within the system's state machine.
- Hardware Integration: A list and brief description of all key hardware components used in the platform. Diagrams illustrating the physical and electrical connections may be included.
- Challenges & Solutions: A discussion of the significant technical, integration, and logistical challenges encountered during the project's development and the approaches taken to resolve or mitigate them.
- Future Improvements: A set of reasoned recommendations for enhancements, additional features, and refinements to be pursued beyond the current project state.

This document is an engineering summary and does not contain exhaustive wiring schematics, step-by-step build instructions, or full code listings. Complete source code, configuration files, and launch scripts are available in the project's GitHub repository.

3 Project Status

The current state of the project represents a significant partial completion of the initial goal. The robot successfully demonstrates full autonomy in the first phase of its mission: it can explore an unknown room, construct a coherent map using Lidar-based SLAM (via SLAM Toolbox and the nav2 stack), and navigate to frontier points using obstacle avoidance.

The vision pipeline for detecting AprilTags is functional. When the robot's Behavior Tree checks for an object detection, the camera node processes images and can identify tags, publishing their pose. This can trigger a transition in the robot's logic, signaling that a target has been sighted.

However, the final retrieval sequence remains incomplete. While the hardware for the arm is integrated and control software has been developed, the autonomous control stack for the arm was not fully developed. Consequently, the robot cannot yet perform the aligned approach, grasping, or exact manipulator actions required to physically retrieve an object. The "return to home" behavior after descovering an AprilTag and moving to be able to pick up the AprilTag object is operational.

In summary, the deliverables for Lidar-based room mapping and object identification via AprilTag detection have been met. The deliverable for autonomous object retrieval via arm control has not been achieved. The robot's operational capability is therefore defined as autonomous exploration and mapping with object detection, but not autonomous physical retrieval.

4 Design Overview

4.1 System Components & Hardware Description

The robot is a complex integration of computational, sensory, and actuation hardware. Power is supplied by three independent 12.2V lithium battery packs for safety and component isolation: one dedicated to the robotic arm, one to the drive motors, and one to a powered USB hub. A 12V-to-5V buck converter ensures safe power delivery to the USB hub. The following components form the physical system:

- **Raspberry Pi 5**: Acts as the primary computational unit, hosting the ROS 2 Jazzy master and all high-level autonomy nodes for navigation, perception, and task execution.
- **Arduino Mega**: Serves as the low-level hardware interface. It receives velocity commands from the ROS navigation stack via serial, translates them into signals for the motor driver, and reads odometry data from the IMU to be relayed back to the Pi.
- **Sabertooth Motor Driver**: A dual-channel driver that receives PWM and direction signals from the Arduino to independently control the two drive motors, enabling skid-steer navigation.
- **Sensors**:
  - **RPLidar A2M8**: A 360-degree scanning LiDAR mounted at the front of the robot. It provides range data for simultaneous localization and mapping (SLAM) and real-time obstacle avoidance.
  - **IMU (BNO08x)**: Provides inertial measurement data. Primarily, the yaw (orientation) data and yaw velocity are fused with wheel odometry through an Extended Kalman Filter(EKF) to improve the accuracy of the robot's estimated pose during mapping and navigation.
  - **Cameras**: Two USB cameras are used. A front-facing navigation camera provides a field of view for initial AprilTag detection. A second camera is mounted on the robotic arm for precise alignment during the intended pickup sequence.

- **PX-100 Robotic Arm**: A 4-degree-of-freedom manipulator intended for object retrieval. It is connected via USB and can be controlled through a dedicated ROS node.
- **Drive Motors & Chassis**: Two IG420049X00048R DC geared motors provide locomotion via a skid-steer tracked chassis. Motor encoders provide basic tick data for crude odometry calculation on the Arduino.

4.2 System Architecture & ROS Nodes

The software system is built on ROS 2 Jazzy and follows a modular node-based architecture. Communication between nodes uses topics, services, and actions. The following are the primary custom nodes developed for this project:

- **Motor_Controller**: This critical node manages serial communication between the Raspberry Pi and the Arduino Mega. It subscribes to the standard `/cmd_vel` (Twist) topic published by the navigation stack, converts the linear and angular velocities into a custom serial packet, and sends it to the Arduino. Conversely, it reads serial packets from the Arduino containing IMU data and motor encoder ticks, republishes the IMU data to the `/imu` topic, and calculates/publishes odometry to the `/odom` topic.
- **apriltag**: A node that processes image streams from the designated camera. It uses the `/apriltag_ros` library to detect AprilTags within the camera's field of view. Upon detection, it publishes the tag's ID and, more importantly, its precise 3D pose (relative position and orientation) to a topic such as `/detections`. This information triggers the robot's high-level mission logic.
- **Behavior_tree**: This node implements the robot's high-level autonomy logic. It coordinates the sequence of operations: initiating exploration, monitoring for tag detections, transitioning to an approach operation when a tag is seen, and commanding a return-to-home behavior. It interacts with other nodes by publishing goals (e.g., to the navigation stack) and subscribing to status updates (e.g., from the tag detector).

---IN PROGRESS---

ADD BEHAVIOR TREE DIAGRAM

- **arm_controller**: (Partially Implemented) This node was designed to autonomously control the PX-100 arm. Its intended function was to subscribe to the AprilTag pose from the arm camera and calculate the joint trajectories necessary to center the gripper over the tag and execute a grasp. In the current implementation, this node is not fully functional; arm operation remains inaccurate with additional logic and sensor use needed.

In addition to these custom nodes, the system heavily utilizes standard ROS 2 packages:

- **nav2 Stack**: Provides all navigation functionalities: SLAM (via slam_toolbox), localization, path planning, and collision avoidance via a costmap.
- **robot_state_publisher**: Publishes the coordinate transforms (TF tree) between the robot's physical components (LiDAR, IMU, base link, cameras).

4.3 Theory of Operation

This section provides a high-level overview of the autonomous workflow, integrating the components and nodes described above.

4.3.1 Top Block Diagram

The system is centered around the Raspberry Pi 5, which coordinates all peripheral devices (Cameras, Arduino, LiDAR, and arm). Figure 1 shows the connections between the the Pi 5 and the directly connected components.

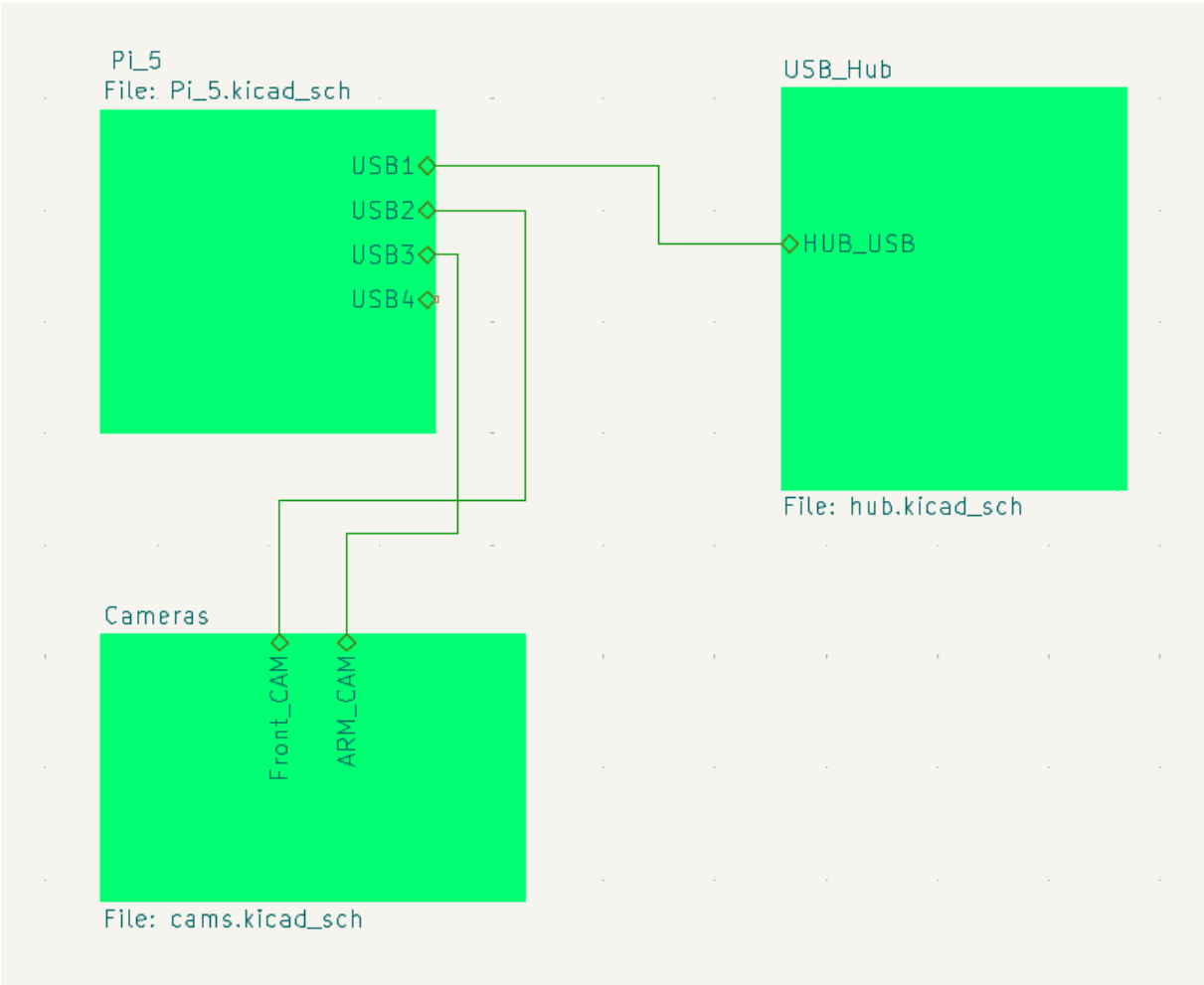

Figure 1: This top block diagram demonstrates the direct connections to the Raspberry Pi 5, see App. A.1 for more diagrams.

4.4 RQT Graph

Figure 2 shows the connections and topics between nodes, publishers, and subscribers.

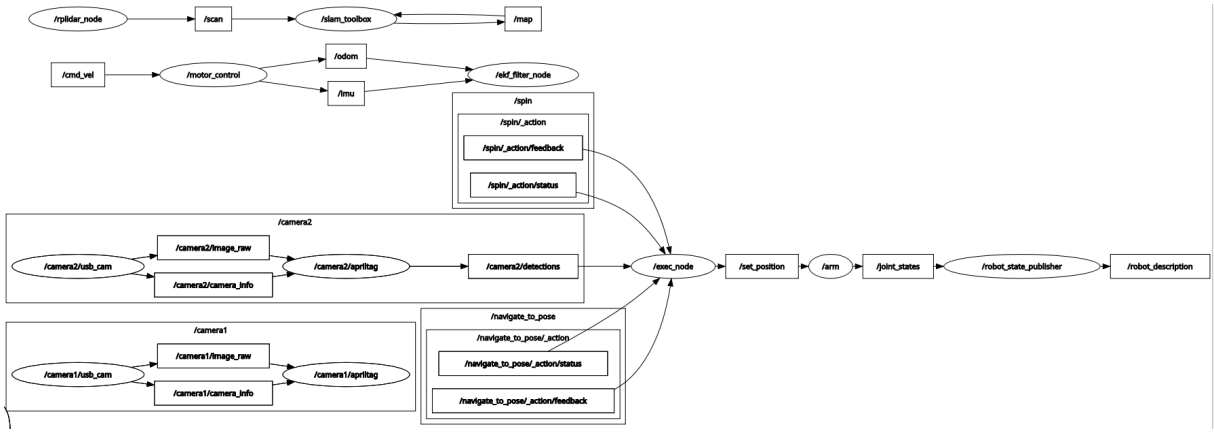

Figure 2: This graph provides an overview of the topics, nodes, publishers, subscirbers, etc. and their connections.

5 Conclusion

This project successfully developed a functional autonomous platform for exploration and mapping with integrated object detection. The robot demonstrates robust capabilities in the initial phases of the intended mission: autonomously navigating an unknown environment using LiDAR-based SLAM, constructing a coherent occupancy grid map, and identifying target objects via AprilTag detection. The integration of a modular ROS 2 architecture with custom hardware interfaces forms a solid foundation for the complete search-and-retrieve system.

5.1 Challenges

The development process presented several significant technical hurdles:

- **AprilTag Detection Reliability**: Consistent detection of AprilTags proved challenging, particularly with smaller tags or at greater distances. Inconsistent lighting, camera focus, and the tag's orientation often led to missed detections, interrupting the autonomous sequence.
- **Motor Control & Speed Tuning**: Translating high-level velocity commands from the `nav2` stack into stable, low-level motor signals via the Arduino connection required extensive tuning. Achieving a balance between responsive navigation and smooth, controlled movement was difficult, with issues such as motor stuttering, uneven speeds.
- **Autonomous Arm Operation**: Integrating the PX-100 arm into the autonomous workflow was the primary unmet challenge. While the hardware was functional, developing the necessary perception-action pipeline—using the arm camera to calculate a precise grasping pose and then generating the corresponding joint trajectories—exceeded the project's time frame. This prevented the completion of the physical retrieval sequence.

5.2 Future Improvements

To advance the system towards full operational capability and robustness, the following improvements are recommended:

- **Camera Calibration**: Performing intrinsic and extrinsic calibration on both cameras. This would rectify lens distortion and accurately determine the camera's position relative to the robot base, as

well as its settings for distance and direction greatly improving the precision of the AprilTag pose estimates and any future computer vision tasks.

- **Complete Arm Integration**: Finalizing the `arm_controller` node is the critical next step. This involves implementing the coordinate transformation logic from camera frame to arm base frame, developing a simple inverse kinematics solver or pre-programmed trajectories for the PX-100, and integrating a grasp execution service into the main behavior tree.
- **Advanced Object Detection**: Moving beyond AprilTags to more generalized object retrieval. This would involve training or implementing a machine learning model (e.g., using ROS 2 integrations with YOLO or TensorFlow) to detect specific classes of objects (e.g., "soda can," "tool"). This would make the system more versatile and applicable to real-world scenarios where objects are not marked with identifiers.
- **Navigation with an Object**: Implementing and testing the "return-to-home" behavior while simultaneously holding a carried object to validate that the robot's dynamics and obstacle avoidance remain stable under altered load conditions.
- **Improved AprilTag Detection**: The current apriltag detection is built into the behavior tree and only checks the /detection topic every 4 seconds. An improved algorithm would run constantly in the background and be accessible to the behavior tree as a service or other node on demand.
- **Calibrated Cost Maps**: Then default costmap settings are far more careful than we need for this project causing the robot to hesitate, retry and fail several times before reaching the desired frontier. The improvement would be calibrating the safe vs. dangerous areas to better fit the dementions of the robot and the desired carefulness.

A Appendix

This section contains supplementary information that supports the main content.

A.1 Circuit Schematics

Complete circuit schematics for the entire design.

A.1.1 Arduino

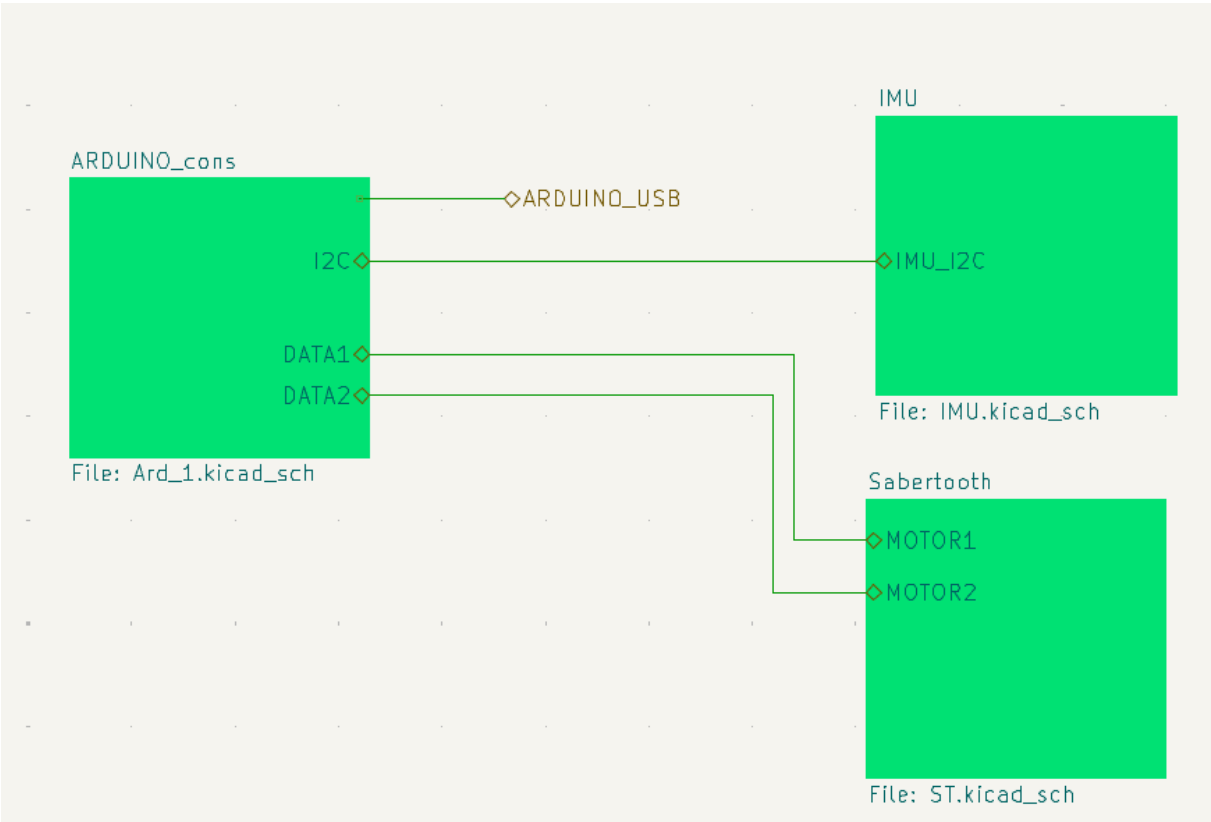

Figure 3: This image shows the connections from the Arduino to the IMU and the Sabertooth motor driver.

A.1.2 USB Hub

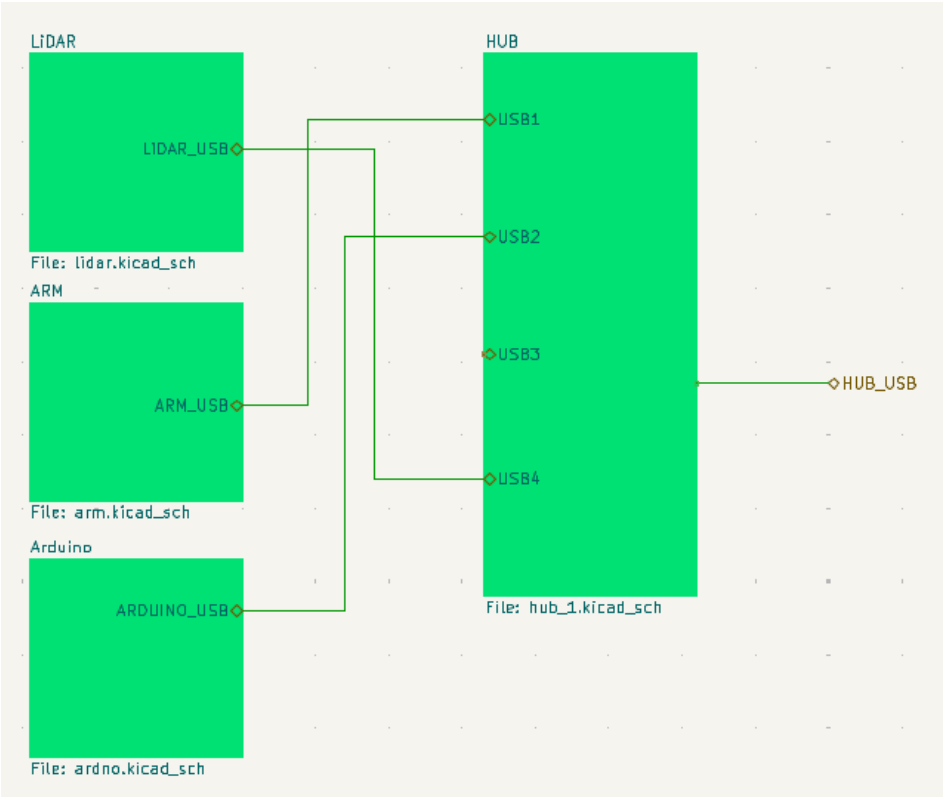

Figure 4: This diagram shows the connections between the USB hub and the LiDAR, the arm, and the Arduino.